04

SKT K-뉴딜 아카데미 · AI·
자동화 기초 · 9/29 오전

정규표현식 탐지 룰

칸이 안 나뉜 줄글에서 필요한 값만 꺼내고,
수상한 계정과 IP 를 찾아냅니다.

1교시 — 각자 필요한 것을 찾아서 공부합니다

오늘은 폴더를 만들고 옮겨 다닙니다. 명령어 정답은 문서에 없습니다.

과제 문서를 엽니다

`0929_tue` → `0929_morning30.md`

오늘 하는 일

- 경로 여섯 낱말을 찾아 정리한다
- `cd` · `..` · `~` · `mkdir` 로 폴더를 만든다
- 공부 기록을 `docs` 안에 파일로 저장한다

> 어제 깃배시를 못 깐 분은 설치부터 합니다. 오늘 다 못 해도 오후 5시 이후에 마무리하면 됩니다.

어제 만든 파일이 드라이브에 있는지 확인합니다

오늘 오전에는 어제 오후에 만든 결과 파일을 다시 씁니다.

파일	어제 어디서	오늘 어디에 쓰나
`log_parser.py` · `agent.log`	오전 4교시	오늘은 안 쓴다
`normalize_logs.py`	오후 7교시	3교시에 정규식으로 다시 만든다
`normalized_logs.json`	오후 7교시	4교시 탐지 룰의 재료

없어도 괜찮습니다. 노트북 맨 위 셀이 필요한 파일을 다시 만들어 줍니다.

로그 20줄을 읽어 딕셔너리 16건과 경보 두 줄을 만듭니다

정규화란 줄글 한 줄을 딕셔너리 한 건으로 바꾸는 일입니다. 로그인 기록이 아닌 4줄은 따로 셉니다 — 16건 + 4건.

```
# 읽을 것 — raw_logs.txt 20줄
2026-09-29 03:11:05 WARN failed login for admin from 211.45.12.9
# 만들 것 ① 줄 하나를 딕셔너리 한 건으로 바꾼다
{'time': '03:11:05', 'level': 'WARN', 'user': 'admin', 'ip':
'211.45.12.9'}
# 만들 것 ② 몇 건인지 세고, 미리 정해 둔 조건에 걸리면 경보를 찍는다 — 4교시
# 그 조건을 룰이라고 합니다. 아래 두 줄이 룰 ①·룰 ② 가 낸 경보입니다

정규화 16건 / 안 맞는 줄 4건
[룰 1] 확인 필요: admin — 실패 4회
[룰 2] 의심 IP: 185.220.101.34 — 계정 4개 시도
```

2교시에 쓸 낱말 세 개를 먼저 짚습니다

re 는 regular expression(정규표현식)의 줄임말이고 파이썬 모듈 이름입니다.

용어 (한글발음)	한 줄 뜻	이 교시에서 보는 모양
비정형 로그 unstructured, 언스트럭처드	값이 안 나뉜 줄글 기록	`09:12:00 WARN failed login for kim01`
정규표현식 regex, 레그엑스	찾을 글자 모양을 규칙으로 적는 표기	\d 숫자 · \w 글자 · + 한 번 이상 → \w+ 는 낱말 하나
메타문자 metacharacter, 메타문자	글자 자체가 아니라 뜻을 갖는 기호	\d \w [ ] + — 이 교시에 쓰는 네 개

오늘부터 읽는 파일은 값이 나뉘어 있지 않습니다

	어제까지 읽은 파일	오늘부터 읽는 파일
한 줄의 모양	`09:01,kim01,LOGIN_OK,10.0.3.21` 쉼표 3개 — 값이 넷으로 갈려 있다	`2026-09-29 09:12:00 WARN failed login for kim01 from 203.0.113.5` 쉼표 0개 — 어디까지가 한 값인지 표시가 없다
값을 가르는 것	쉼표가 값 넷을 갈라 놓았습니다	값을 가르는 기호가 없습니다. 한 줄이 사람이 읽는 문장 형태로 이어집니다
누가 만들었나	사람이 표로 정리해 둔 파일입니다	방화벽·웹서버가 남긴 기록을 그대로 받은 것입니다

공백으로 잘라 「몇 번째 값」으로 세면, 줄 모양이 조금만 달라져도 에러 없이 틀린 값이 들어옵니다.

쉼표가 없으면 어제 쓰던 `split(",")` 가 값을 하나도 나누지 못합니다

방화벽·웹서버는 사람이 읽기 좋게 문장처럼 남깁니다. 그래서 어제 배운 「몇 번째 값」 방식이 안 통합니다.

```
# 어제 — CSV, 칸이 나뉘어 있다
09:01,kim01,LOGIN_OK,10.0.3.21
# 오늘 — 방화벽·서버가 남긴 줄글 (쉼표 0개)
2026-09-29 09:12:00 WARN failed login for kim01 from 203.0.113.5

# 어제 방식을 그대로 써 보면
print(line.split(","))
```

```
['2026-09-29 09:12:00 WARN failed login for kim01 from 203.0.113.5']
— 자를 데가 없어 한 덩이 그대로. parts[3] 은 IndexError
```

공백으로 자르면 값 아홉 개가 되고 계정은 `parts[6]` 입니다

`split()` 은 공백마다 자릅니다. 0 부터 세어 날짜 · 시각 · 등급 · accepted · login · for · 6번이 계정.

```
line = "2026-09-29 09:02:11 INFO accepted login for kim.cs from 10.1.2.11"
parts = line.split()
print(parts)
```

```
['2026-09-29', '09:02:11', 'INFO', 'accepted', 'login', 'for', 'kim.cs',
'from', '10.1.2.11']
```

```
print(parts[6])
```

```
kim.cs
```

공백 기준으로 나눴을 때 조용한 실패가 된다. 같은 인덱스가 다른 값을 꺼내는데도 에러도 안남

`invalid user` 가 끼면 인덱스가 두 칸씩 달라져 계정은 인덱스 8에 놓입니다.

값이 아홉 개

```
line = "2026-09-29 09:02:11 INFO accepted login for kim.cs from 10.1.2.11"
parts = line.split()
print(parts[6])
```

```
kim.cs
```

값이 열한 개

```
line = "2026-09-29 23:40:07 WARN failed login for invalid user guest from 185.220.101.34"
parts = line.split()
print(parts[6])
```

```
invalid
```

고정된 인덱스 대신 글자 패턴으로 값을 찾습니다

위치로 찾기 — `split`

「인덱스 6」

앞에 낱말이 하나 늘면 전부 밀린다

틀려도 에러가 안 난다

패턴으로 찾기 — 정규표현식

「`login for` 뒤에 붙은 낱말」

줄의 길이가 달라도 상관없다

안 맞으면 못 찾았다고 알려 준다

re.findall 은 맞는 것을 전부 목록으로 줍니다

패턴 앞의 r 은 역슬래시를 그대로 쓰겠다는 표시입니다. 빼면 파이썬이 \d 를 먼저 해석하려다 경고를 냅니다.

```
import re
print(re.findall(r"\d", "a1b2"))
```

```
['1', '2']
```

적는 자리	무엇을 적나
첫째 자리	찾을 패턴 — r"\d" (\d는 숫자 1개라는 의미)
둘째 자리	패턴을 찾을 대상 글자 — "a1b2"

\d · \w · . 는 각각 글자 한 개를 가리킵니다

앞의 \ 는 역슬래시(한글 자판의 ₩ 자리). . 는 아무 글자나 한 개라서 진짜 점은 \. 로 적습니다.

기호	무엇 한 개	돌려 본 결과
\d	숫자 한 개	re.findall(r"\d", "a1b2") → ['1', '2']
\w	글자·숫자·밑줄 한 개	re.findall(r"\w", "a1_!") → ['a', '1', '_']
.	아무 글자나 한 개	re.findall(r".", "a1_") → ['a', '1', '_']

\d 와 \w 가 어디까지를 숫자·글자로 보는지 정해 둡니다

기준은 파이썬이 정해 둔 목록입니다. 우리가 고르는 것이 아니라 이미 정해져 있습니다.

기호	이것까지 맞다	이것은 안 맞는다 — 여기서 끊긴다
\d	0 부터 9 까지 — 그뿐입니다	점 . · 붙임표 - · 콜론 : · 빈칸 · 영문자
\w	영문자 a~z A~Z · 숫자 0~9 · 밑줄 _ 한글도 \w 에 들어갑니다 re.findall(r"\w+", "홍길동 kim01") → ['홍길동', 'kim01']	점 . · 붙임표 - · 콜론 : · 빈칸 · @ · /

오늘 덩어리가 끊기는 자리는 전부 오른쪽 칸 때문입니다. 점·붙임표·콜론·빈칸은 숫자도 글자도 아닙니다. 그래서 그것까지 붙이려면 [\d.] 처럼 대괄호에 직접 넣어 줍니다.

+ 는 왼쪽 기호를 1개 이상 이어 붙입니다

> + 는 왼쪽 기호를 이어 붙이다가 맞지 않는 글자에서 멈춥니다. 기준은 공백이 아닙니다.

패턴	이 패턴을 읽는 법	찾아볼 글자와 돌려 본 결과
`\w`	글자 한 개 이어 붙이지 않으므로 한 글자마다 따로 잡는다	`re.findall(r"\w", "kim01 from")` → `['k', 'i', 'm', '0', '1', 'f', 'r', 'o', 'm']`
`\w+`	+ 의 왼쪽에 적힌 `\w` 를 1개 이상 이어 붙인다 공백은 `\w` 가 아니므로 공백에서 멈춘다	`re.findall(r"\w+", "kim01 from")` → `['kim01', 'from']`

`\d+` 는 숫자가 이어지는 만큼 한 덩어리로 묶고, 숫자가 아닌 글자에서 덩어리를 끊습니다

`re.findall(r"\d+", "2026-09-29 09:12:00")` 을 왼쪽부터 한 글자씩 따라가 봅니다.

2026	-	09	-	29	공백	09	:	12	:	00
맞음	멈춤	맞음	멈춤	맞음	멈춤	맞음	멈춤	맞음	멈춤	맞음

```
import re
print(re.findall(r"\d+", "2026-09-29 09:12:00"))
```

```
['2026', '09', '29', '09', '12', '00']
```

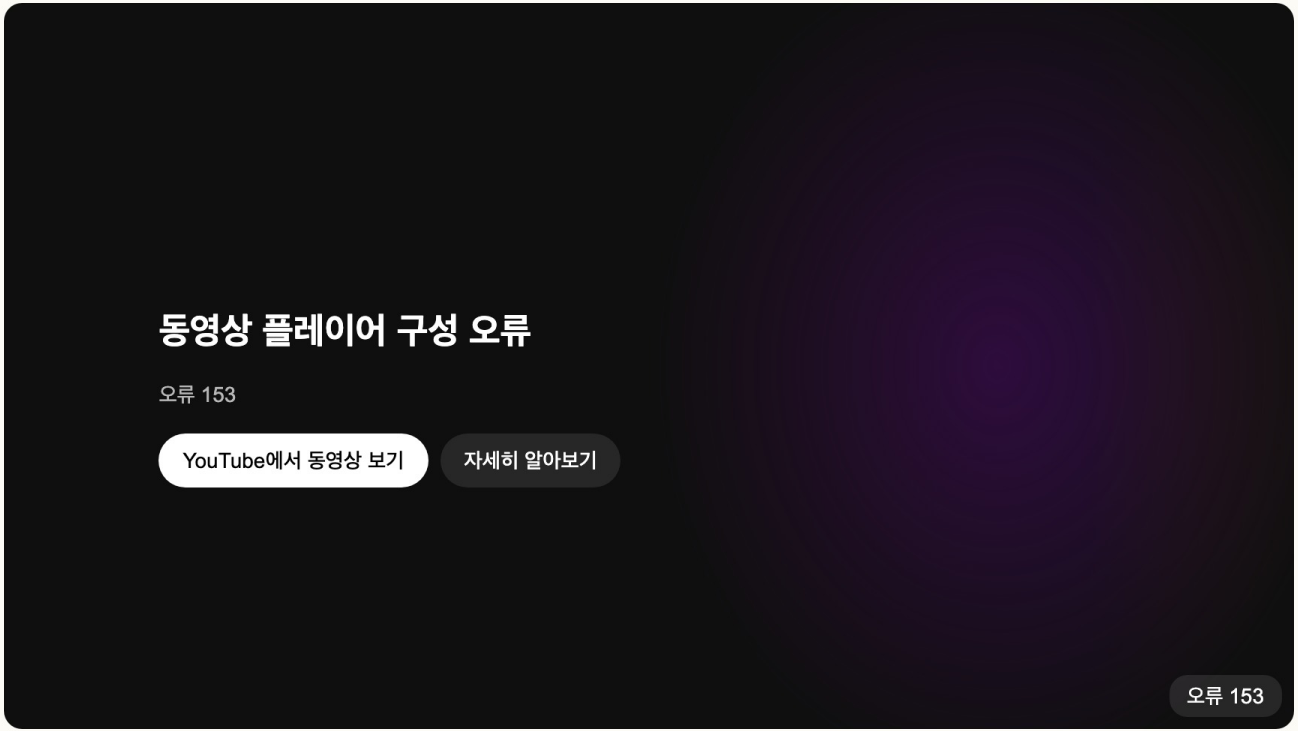
숫자가 아닌 글자(붙임표·빈칸·콜론)를 만나면 덩어리가 거기서 끝나고, 다음 숫자에서 새 덩어리가 시작됩니다. 그래서 여섯 덩어리입니다.

대괄호는 그 안의 글자 중 아무거나를 가리킵니다

`[\w.]` 는 글자 또는 점 한 개라는 뜻입니다. 점을 안에 넣어 두었으니 점에서 멈추지 않습니다.

패턴	이 패턴을 읽는 법	찾아볼 글자와 돌려 본 결과
`\w+`	글자만 이어 붙인다 점은 `\w` 가 아니므로 **점에서 멈춘다**	`re.findall(r"\w+", "kim.cs from")` → `['kim', 'cs', 'from']`
`[\w.]+`	`+` 의 왼쪽에 적힌 `[\w.]` 즉 **글자 또는 점을 1개 이상** 이어 붙인다	`re.findall(r"[\w.]+", "kim.cs from")` → `['kim.cs', 'from']`

4분 영상으로 정규표현식 기호를 한 번 더 봅니다



2교시에 배운 것을 정리합니다

쓰는 법	뜻
`line.split()`	공백으로 자른다. 값의 인덱스가 달라지면 조용히 틀린다
`import re`	정규표현식 도구를 부른다
`\d \w [ ] +`	오늘 코드에 쓰는 기호 네 개.

> 기호는 이 네 개만 씁니다.

실습 ① · 노트북 2교시 절 · 01_0929_am_regex_detection_rules.ipynb

문제 2-1~2-7 · 도전 ⭐2-6 · ⭐2-7

① **사본을 저장하고 맨 위 셀을 실행합니다**

`raw_logs.txt` 20줄이 만들어집니다

② **문제 2-1·2-2 는 실행하기 전에 답을 적습니다**

`parts[6]` 에 `invalid` 가 섞여 나오는 것을 직접 맞혀 봅니다

③ **문제 2-3~2-5 와 도전 ⭐2-6·⭐2-7**

파일 전체에서 인덱스 6의 값이 계정이 아닌 줄을 찾아내면 성공

대괄호 안에는 이어 붙일 글자를 직접 골라 넣습니다

[ ] 는 안에 적은 것 중 아무거나 한 개, 오른쪽의 + 는 그것을 1개 이상 이어 붙인다는 뜻입니다.

패턴	대괄호에 무엇을 넣었나	찾아볼 글자와 돌려 본 결과
`[\w.]+`	\w 와 점 → 글자 또는 점	`re.findall(r"[\w.]+", "kim.cs from")` → `['kim.cs', 'from']`
`[\d.]+`	\d 와 점 → 숫자 또는 점	`re.findall(r"[\d.]+", "203.0.113.5")` → `['203.0.113.5']`
`[\d:]+`	\d 와 콜론 → 숫자 또는 콜론	`re.findall(r"[\d:]+", "09:12:00 WARN")` → `['09:12:00']`
`[\d.]+`	대괄호에 붙임표(-)는 안 넣었다	`re.findall(r"[\d.]+", "2026-09-29")` → `['2026', '09', '29']` — 붙임표에서 끊긴다

IP 를 한 덩어리로 잡으려면 점도 대괄호에 넣습니다

오늘 로그의 IPv4 주소는 숫자와 점으로 되어 있습니다. `[\d.]+` 는 주소를 검증하지 않고 그 모양의 글자를 찾습니다.

```
import re
ip = "203.0.113.5"
print(re.findall(r"\d+", ip))
```

```
['203', '0', '113', '5']
```

```
print(re.findall(r"[\d.]+", ip))
```

```
['203.0.113.5']
```

\d
숫자 한 글자

\d+
숫자 한 글자 이상이 연속된 덩어리

re.search 는 찾은 결과를 m 에 담습니다

re.search(패턴, 찾아볼 글자) 는 맞는 첫 부분 하나를 찾아 돌려줍니다.
그 결과를 m 이라는 변수에 담았습니다 — m이라는 이름은 아무거나 지어도 됩니다.

```
import re
ip = "203.0.113.5"
m = re.search(r"[\d.]+", ip)  # 숫자 또는 점이 이어진 만큼
print(m)
```

```
<re.Match object; span=(0, 11), match='203.0.113.5'>
```

print(m) 을 하면 글자가 바로 나오지 않습니다. 두 가지가 함께 적혀 나옵니다.
span=(0, 11) 는 몇 번째 글자에서 찾았는지, match= 뒤는 찾은 글자입니다.

찾은 글자만 쓰고 싶을 때 `m.group()` 으로 꺼냅니다

m 은 찾은 자리와 글자가 함께 적힌 결과입니다. 그래서 찾은 글자만 필요할 때 — 파일에 적거나 딕셔너리에 담을 때 — `.group()` 을 붙입니다.

```
import re
ip = "203.0.113.5"
m = re.search(r"[\d.]+", ip)  # 숫자 또는 점이 이어진 만큼
print(m.group())
```

```
203.0.113.5
```

`print(m)`	자리와 글자가 함께 적힌 결과 전체가 나옵니다
`print(m.group())`	찾은 글자만 나옵니다 — `203.0.113.5`

원하는 값 앞의 고정된 글자를 찾는 단서로 씁니다

`from` 은 파이썬 명령어가 아니라 로그에 실제로 적힌 말입니다. IP 바로 앞에 늘 나오므로 표지판처럼 이용합니다.

찾는 패턴	어디에서 처음 맞나
`[\d.]+`	줄 앞의 날짜가 먼저 조건에 맞아 `2026`이 잡힙니다
`from [\d.]+`	`from` 뒤의 숫자·점 덩어리를 찾으므로 IP 쪽으로 바로 가서 `from 203.0.113.5`가 잡힙니다

지금은 `from`도 결과에 포함됩니다. 이후 그룹을 배우면 `203.0.113.5`만 떼어 냅니다.

IP 앞의 `from` 까지 함께 찾으면 날짜를 건너뜁니다

`re.search` 는 줄의 맨 앞부터 훑다가 처음 맞는 곳에서 멈춥니다. 그래서 `[\d.]+` 만 적으면 날짜의 2026 이 먼저 걸립니다.

```
import re
line = "2026-09-29 09:12:00 WARN failed login for kim01 from 203.0.113.5"
print(re.search(r"[\d.]+", line).group())
```

```
2026
```

```
print(re.search(r"from [\d.]+", line).group())
```

```
from 203.0.113.5
```

`r"[\d.]+"`	줄 맨 앞부터 훑다가 날짜의 2026 에서 먼저 맞아 버린다
`r"from [\d.]+"`	`from` 이라는 글자가 먼저 나와야 맞으므로 날짜는 건너뛰고 IP 에서 맞는다

search 는 처음 맞는 하나, findall 은 맞는 것 전부를 찾습니다

re.search
처음 맞는 하나

```
re.search(...).group()
→ 'from 203.0.113.5'
```

re.findall
맞는 것 전부

```
re.findall(...)
→ ['10', '1', '2', '11']
```

> 하나를 찾을 때는 search, 같은 모양을 모두 모을 때는 findall을 씁니다.

그룹에 이름을 붙이면 딕셔너리로 꺼낼 수 있습니다

① 그룹

(패턴)

꺼낼 부분을 묶는다

→

② 이름 붙이기

(?P<user>패턴)

user라고 부른다

→

③ 딕셔너리

m.groupdict()

{'user': 'kim01'}

패턴 하나로 시각 · 등급 · 계정 · IP 를 한 번에 꺼냅니다

지금까지는 `re.search` 로 처음 맞는 덩어리 하나를 꺼냈습니다. 로그 한 줄에서 꺼낼 값은 넷입니다.

어떻게 꺼내나	그러면 어떻게 되나
값마다 따로 찾는다 ① 시각 · ② 등급 · ③ 계정 · ④ IP `re.search(...)` 를 네 번 실행	틀린 방법은 아니지만 같은 줄을 네 번 훑고, 로그 모양이 바뀌면 **네 군데를 고쳐야** 합니다
패턴 하나에 받고 싶은 부분 앞뒤에 소괄호를 적는다 `r"for ([\w.]+) from ([\d.]+)"`	줄을 한 번만 훑고, 괄호 안에 든 부분만 **따로 받을 수 있습니다** 소괄호는 비유가 아니라 그대로 치는 글자이고, 그 괄호를 **그룹**이라고 합니다

(?P<이름>패턴)은 이름 있는 캡처 그룹입니다

파이썬 re에서 패턴의 일부를 따로 저장하고 이름으로 꺼낼 때 사용하는 공식 문법입니다.

```
(?P<user>[\w.]+)
```

( )
일치한 글자를 저장하는
캡처 그룹

?P<user>
대문자 P: 파이썬 표기
user: 그룹 이름

[\w.]+
그룹이 실제로 찾는
정규식 패턴

m.group("user")로 이 그룹의 값 하나를 꺼내고, m.groupdict()로 이름 있는 그룹 전체를 딕셔너리로 꺼냅니다.

긴 패턴도 왼쪽부터 다섯 조각으로 읽습니다

```
# 패턴으로 읽을 원본 로그 문자열
2026-09-29 09:12:00 WARN failed login for kim01 from 203.0.113.5
```

`(?P<time>[\d:]+)`	숫자·콜론 덩어리 → `time = 09:12:00`
`(?P<level>\w+)`	글자 덩어리 → `level = WARN`
`\w+ login for`	`failed login for`가 있어야 맞음 · 값으로는 안 꺼냄
`(?P<user>[\w.]+)`	글자·점 덩어리 → `user = kim01`
`from (?P<ip>[\d.]+)`	`from` 뒤 숫자·점 덩어리 → `ip = 203.0.113.5`

패턴 사이의 빈칸도 로그의 빈칸과 맞아야 합니다.

꺼낼 자리를 괄호로 묶고 그 괄호에 이름을 적습니다

모양은 늘 같습니다 — `(?P<이름>패턴)`.

```
import re
line = "2026-09-29 09:12:00 WARN failed login for kim01 from 203.0.113.5"
m = re.search(r"(?P<time>[\d:]+) (?P<level>\w+) \w+ login for (?P<user>[\w.]+) from (?P<ip>[\d.]+)", line)

print(m.group("user"))
```

```
kim01
```

괄호 안 네 곳	**값으로 꺼내진다** — 이름은 `time` · `level` · `user` · `ip`
괄호 밖 — `login for` · `from`	줄에 있어야 맞지만 값으로는 안 나온다
`m.group("user")`	그룹 인덱스가 아니라 **그룹 이름으로 꺼낸다** — 읽으면 무엇인지 안다

`.groupdict()` 는 줄 하나를 딕셔너리 하나로 바꿉니다

이 네 그룹 이름 time · level · user · ip 가 오늘 실습의 평가 기준입니다.

```
import re
line = "2026-09-29 09:12:00 WARN failed login for kim01 from
203.0.113.5"
PATTERN = r"(?P<time>[\d:]+) (?P<level>\w+) \w+ login for (?
P<user>[\w.]+) from (?P<ip>[\d.]+)"
m = re.search(PATTERN, line)
print(m.groupdict())
```

```
{'time': '09:12:00', 'level': 'WARN', 'user': 'kim01', 'ip':
'203.0.113.5'}
```

3교시에 배운 것을 정리합니다

쓰는 법	뜻
`re.search(r"패턴", 글자).group()`	맞는 첫 부분을 찾아 글자를 꺼낸다
`(?P<time>[\d:]+)`	꺼낼 자리를 괄호로 묶고 이름을 붙인다
`.groupdict()`	이름과 값을 짝지어 딕셔너리로 준다
`m.group("user")`	그룹 인덱스가 아니라 그룹 이름으로 꺼낸다. 못 찾았을 때 어떻게 하는지는 4교시에서 본다

실습 ② · 노트북 3교시 절

문제 3-1~3-7 · 도전 ⭐3-7

① **문제 3-1~3-3 — 패턴으로 찾기**

`from` 뒤의 IP 와 `for` 뒤의 계정을 뽑아내면 성공

② **문제 3-4·3-5 — 이름 붙인 괄호로 꺼내기**

`.groupdict()` 로 딕셔너리 한 개가 나오면 성공

③ **문제 3-6 · 도전 ⭐3-7 — 파일 전체를 딕셔너리로**

정규화 `16건` 이 나오면 성공

\w+ login for는 failed login for를 찾습니다

```
import re
line = "2026-09-29 09:12:00 WARN failed login for kim01 from 203.0.113.5"
PATTERN = r"(?P<time>[\d:]+) (?P<level>\w+) \w+ login for (?P<user>[\w.]+)
from (?P<ip>[\d.]+)"
m = re.search(PATTERN, line)
print(m.groupdict())
```

```
{'time': '09:12:00', 'level': 'WARN', 'user': 'kim01', 'ip':
'203.0.113.5'}
```

\w+ → failed
글자 덩어리 하나를 찾습니다

login for → login for
기호가 아니라 그대로 일치해야 하는 문자열입니다

PATTERN이 요구하는 순서가 없으면 None 이 나옵니다

PATTERN이 찾는 순서: 등급 → failed login for → 계정 → from → IP
실제 줄의 순서: 등급 → session closed for → IP
failed login for 계정 from 구간이 없으므로 패턴 전체가 맞지 않습니다.

```
line = "2026-09-29 09:16:02 INFO session closed for 10.1.2.34"
print(re.search(PATTERN, line))
```

```
None
```

```
# 그대로 값을 꺼내려 하면
print(re.search(PATTERN, line).group())
```

```
AttributeError: 'NoneType' object has no attribute 'group'
```

패턴에 안 맞은 줄을 `unmatched_logs.txt` 에 남깁니다

그냥 건너뛰면 코드는 깔끔하지만 데이터가 조용히 사라집니다.
안 맞은 줄은 `unmatched_logs.txt` 에 남깁니다.

```
rows = []
unmatched = []
for line in open("raw_logs.txt"):
    m = re.search(PATTERN, line)
    if m:
        rows.append(m.groupdict())
    else:
        unmatched.append(line.strip())
```

정규화 16건 / 안 맞는 줄 4건

normalize_logs.py 를 정규식으로 다시 씁니다

어제 도전 문제: parse_line(line)이 딕셔너리를 return
오늘: parse_raw_logs(line)가 맞으면 딕셔너리, 안 맞으면 None을 return
return은 함수가 만든 결과를 함수 밖으로 돌려주는 명령입니다.

```
PATTERN = r"(?P<time>[\d:]+) (?P<level>\w+) \w+ login for (?P<user>
[\w.]+) from (?P<ip>[\d.]+)"

def parse_raw_logs(line):

    m = re.search(PATTERN, line)
    if m:
        return m.groupdict()
    return None
# 파일을 돌며 부르고, None 이면 unmatched 로. 끝에 json.dump 로 저장
```

탐지 룰은 수상하다고 판단할 조건을 코드로 적은 것입니다

로그는 무슨 일이 있었는지 기록합니다. 탐지 룰은 그 기록 중 확인이 필요한 것을 같은 기준으로 자동으로 고릅니다.

① 기록
`normalized_logs.json`

→

② 조건
계정별 실패 횟수를 세고
3회 이상인지 확인

→

③ 경보
admin을 대상으로
실패 4회라는 근거도 출력

오늘의 룰 ①: 한 계정에 실패가 반복됨 · 룰 ②: 한 IP가 여러 계정을 시도함

탐지 룰 하나를 만들 때 무엇을 · 어떻게 · 몇부터 를 정합니다

정할 것	무엇을 정하나	룰 ①의 예
① 무엇을 세나	무엇을 키로 두느냐	계정별 실패 횟수
② 몇부터 수상한가	임계값 — 사람이 정한다	`THRESHOLD = 3`
③ 무엇을 남기나	룰 번호와 근거 숫자	`[룰 1] … 실패 4회`

브루트포스는 비밀번호를 맞을 때까지 반복 대입하는 공격입니다

> brute force · 무차별 대입. 한 계정에 로그인 실패가 쌓이면 사람이 아니라고 봅니다.

```
# raw_logs.txt 의 실제 줄 — 6분 사이 다섯 줄
03:11:05 WARN failed   login for admin from 211.45.12.9
03:12:47 WARN failed   login for admin from 211.45.12.9
03:13:58 WARN failed   login for admin from 211.45.12.9
03:15:22 WARN failed   login for admin from 211.45.12.9
03:17:09 INFO accepted login for admin from 211.45.12.9
```

같은 계정인가	다섯 줄 다 `admin`
실패가 몇 번인가	`WARN failed` 가 4회 — 이것이 룰 ①의 기준

첫 번째 탐지 규칙은 브루트포스입니다

브루트포스는 한 계정의 비밀번호를 바꿔 가며 반복해서 로그인하는 공격입니다. 따라서 실패 기록이 한 계정에 집중됩니다.

① 실패만 선택
`level == "WARN"`

→

② 계정별 집계
계정: 실패 횟수

→

③ 기준과 비교
3회 이상이면 확인

계정을 키로 두고 처음이면 1, 이미 있으면 1을 더합니다. 지난주에 배운 `Counter`를 사용해도 같은 결과가 나옵니다.

브루트포스 탐지의 첫 단계로 실패한 기록만 셉니다

계정별로 나누기 전에, 실패가 몇 건인지부터 봅니다.

16건 중 10건이 실패입니다.
그런데 이 숫자만으로는 누가 위험한지 모릅니다 — 그래서 계정별로 나눠 셉니다.

```
rows = json.load(open("normalized_logs.json", encoding="utf-8"))
fail_count = 0
for row in rows:
    if row["level"] == "WARN":
        fail_count = fail_count + 1
print(f"실패 {fail_count}건")
```

```
실패 10건
```

fail에 계정별 WARN 횟수를 저장합니다

fail의 모양은 {계정: WARN 횟수}입니다. THRESHOLD = 3은 다음 장에서 이 횟수와 비교할 기준입니다.

```
rows = json.load(open("normalized_logs.json", encoding="utf-8"))
THRESHOLD = 3
fail = {}
for row in rows:
    if row["level"] == "WARN":
        user = row["user"]
        if user in fail:

            fail[user] = fail[user] + 1
        else:
            fail[user] = 1
print(fail)
```

```
{'admin': 4, 'kim01': 1, 'park.js': 1, 'kim.cs': 1,
 'lee.yh': 1, 'choi.mk': 1, 'jung.hw': 1}
```

계정별 WARN 횟수에서 admin만 3회 이상입니다

계정	실패	기준 3 과 견주면
admin	4회	걸린다
kim01 · park.js · kim.cs	각 1회	경보 없음
lee.yh · choi.mk · jung.hw	각 1회	경보 없음

실패 10건이 일곱 계정에 흩어진 게 아니라 한 계정에 몰려 있습니다.
총합만 봤을 때는 안 보이던 것입니다.

룰 ①은 기준을 넘은 계정에 경보 한 줄을 찍습니다

어제 만든 「확인 필요: admin — 실패 3회」와 **거의 같은 줄**입니다.
어제는 CSV 에서, 오늘은 줄글에서 왔을 뿐 — 데이터만 바뀌었습니다.

```
fail = {"kim01": 1, "admin": 4}    # 앞 장에서 센 결과
THRESHOLD = 3
for user in fail:
    if fail[user] >= THRESHOLD:
        print(f"[룰 1] 확인 필요: {user} — 실패 {fail[user]}회")
```

```
[룰 1] 확인 필요: admin — 실패 4회
```

룰 ②는 계정이 아니라 IP 를 기준으로 봅니다

룰 ① — 계정을 기준으로

- 「한 계정에 실패가 몰렸나」
- admin — 4회 → 걸린다
- kim.cs · lee.yh 는 각각 1회 → 안 걸린다

룰 ② — IP 를 기준으로

- 「한 곳에서 여러 계정을 건드렸나」
- 185.220.101.34 — 계정 4개 → 걸린다
- 계정마다 1회씩이라 룰 ①에는 안 걸렸다

이 코드는 IP별로 서로 다른 계정을 모읍니다

> 알아내려는 것은 실패 횟수가 아니라 한 IP가 건드린 서로 다른 계정의 수입니다.

① **로그 한 줄에서 두 값을 읽습니다**

`row["ip"]` · `row["user"]`

② **같은 IP에서 나온 기록을 한곳에 묶습니다**

딕셔너리의 키는 `ip`입니다

③ **서로 다른 계정만 리스트에 남깁니다**

```
["kim.cs", "lee.yh", "choi.mk", "jung.hw"]
```

IP 마다 시도된 계정을 리스트로 모읍니다

```
rows = json.load(open("normalized_logs.json", encoding="utf-8"))
table = {}
for row in rows:
    ip, user = row["ip"], row["user"]
    if ip in table:
        if user not in table[ip]:
            table[ip].append(user)
    else:
        table[ip] = [user]
```

```
{'203.0.113.5': ['kim01'], '10.1.2.11': ['kim.cs'],
 '185.220.101.34': ['kim.cs', 'lee.yh', 'choi.mk', 'jung.hw']}
```

룰 ②는 계정이 둘 이상인 IP 에 경보를 냅니다

기준을 2 로 정했습니다 — 한 사람이 남의 계정을 잘못 치는 일은 드물기 때문입니다.
다만 **공용 PC 나 사무실 공유기**라면 여러 사람이 같은 IP 를 써서 오탐이 납니다. 그때는 기준을 올립니다.

```
table = {"203.0.113.5": ["kim01"],
         "185.220.101.34": ["kim.cs", "lee.yh", "choi.mk",
"jung.hw"]}
for ip in table:
    if len(table[ip]) >= 2:
        print(f"[룰 2] 의심 IP: {ip} — 계정 {len(table[ip])}개 시도")
```

```
[룰 2] 의심 IP: 185.220.101.34 — 계정 4개 시도
```

오전 산출물 세 개를 드라이브에 남깁니다

파일	무엇	어디에 쓰나
`normalize_logs.py`	줄글을 딕셔너리로 바꾸는 프로그램	오후에 룰 ③ 을 이어 붙인다
`normalized_logs.json`	정규화한 16건	오후 탐지 룰의 재료
`unmatched_logs.txt`	안 맞은 4줄	왜 안 맞았는지 나중에 확인

> `%cd` 로 `agent_core` 로 옮긴 뒤 셀을 다시 실행합니다. 어제와 같은 순서입니다.

4교시에 배운 것을 정리합니다

쓰는 법	뜻
`if 이름 in 딕셔너리:`	이미 담겨 있는지 확인한다
`딕셔너리[키] = 값 + 1`	세기
`딕셔너리[키] = [값]`	값 자리에 리스트를 둔다
`THRESHOLD`	몇 번부터 수상한지는 사람이 정한다

오전 산출물은 `normalize_logs.py` · `normalized_logs.json` · `unmatched_logs.txt` 셋.
오후에 룰 ③ 을 더합니다.

여유가 있으면 ⭐도전 문제를 풉니다

도전	무엇을 해 보나
⭐2-6 계정이 아닌 줄 세기	고정 인덱스로 꺼내면 왜 틀리는지 직접 확인
⭐3-5 그룹 순서 바꿔 보기	그룹 인덱스가 괄호 순서를 따라간다는 것을 확인
⭐4-10 두 룰을 한 번에	룰 ①②를 이어 붙여 경보 두 줄 — 오후에 룰 ③ 을 여기 더합니다

기본 문제를 못 끝냈으면 도전이 아니라 기본부터입니다. 오후 8교시에 모아 푸는 시간이 있습니다.

실습 ③ · 노트북 4교시 절

문제 4-1~4-14 · 도전 ⭐4-4 · ⭐4-10 · ⭐4-14

① **문제 4-1~⭐4-4 — 안 맞는 줄과 `normalize_logs.py`**

안 맞음 4건 이 나오고 파일이 돌면 성공

② **문제 4-5~4-10 — 룰 ①**

`[룰 1] 확인 필요: admin — 실패 4회` 가 나오면 성공

③ **문제 4-11~⭐4-14 — 룰 ② 와 드라이브**

`[룰 2] 의심 IP: 185.220.101.34 — 계정 4개 시도` 가 나오면 성공